

Ontologies, semantic layers and knowledge graphs

Executive summary

An **ontology** (in computer/information science) is a **formal, explicit specification of a conceptualisation** that is shared by a community or organisation, typically intended to enable **reuse, interoperability, and automated reasoning**. The most-cited foundational definitions emphasise (i) a shared **vocabulary** of concepts and relations and (ii) a **formal** (machine-interpretable, logic-based) specification that supports analysis and computation. ¹

In practice, ontologies are rarely used alone: they sit in an ecosystem of standards and artefacts. The **W3C Semantic Web stack** provides core languages for representing and querying knowledge: **RDF** (graph data model), **RDFS** (lightweight schema vocabulary), **OWL/OWL 2** (ontology language with formal semantics, profiles, and reasoning support), **SKOS** (model for thesauri/taxonomies and “knowledge organisation systems”), and **SPARQL** (query language for RDF graphs). ²

Knowledge graphs (KGs), semantic layers, and ontologies overlap but are not interchangeable. A KG is typically a **graph of entities/concepts and typed relations** (often with provenance and context) used for integration, search, analytics, and reasoning; an ontology often acts as the **schema/semantic backbone** for a KG (or for data access/integration via mappings). Semantic layers in BI, by contrast, mainly provide a **business-friendly abstraction** of data (dimensions, measures, metric definitions), and may or may not use formal logic; however, ontology-based approaches (e.g., OBDA) can also be viewed as a semantic layer that enables **conceptual querying over heterogeneous sources**. ³

Recent NLP and LLM methods shift the construction of ontologies and KGs from purely manual engineering to **hybrid pipelines**: automated extraction (entities/relations), schema induction and ontology learning, alignment/mapping across sources, and rigorous validation (constraints + reasoning + human review). There is a growing research literature characterising how LLMs reshape classical KG pipelines, while also highlighting the need for evaluation and guardrails (e.g., structured outputs, constraint checking, sampling, and audit trails). ⁴

Ontology fundamentals

What an ontology is

A widely cited definition describes an ontology as **a specification of a representational vocabulary for a shared domain of discourse**, including definitions of classes and relations. ⁵ Another mainstream definition merges earlier views into: **“a formal, explicit specification of a shared conceptualisation”**. ⁶ These definitions are compatible: they position an ontology as (a) a **shared conceptual model** and (b) a **formal artefact** that can be processed by machines. ⁷

In formal terms, many ontologies are grounded in **logic-based knowledge representation** (notably description logics, which underpin OWL profiles and reasoning). ⁸ This “logic grounding” is what

differentiates an ontology from a purely informal glossary: it enables automated inference such as classification and consistency checking (subject to the chosen language/profile). ⁹

Common types of ontology

Ontology “type” is partly about **scope** and partly about **logical strength**:

By scope and intent - Upper/top ontologies: very general categories intended for broad reuse across domains (e.g., notions like object, process, role). Upper ontologies are often used to improve interoperability and alignment across domain models. ¹⁰

- **Domain ontologies:** concepts and relations for a specific domain (e.g., finance, manufacturing, clinical data). ¹¹

- **Application/task ontologies:** tailored to a particular application or task (e.g., decision support, configuration), often reflecting operational constraints and required queries. ¹²

By expressivity - Lightweight ontologies: mostly taxonomic structure and simple constraints (often close to RDFS/SKOS). ¹³

- **More expressive (heavyweight) ontologies:** richer axioms (disjointness, equivalence, restrictions, property characteristics, etc.) typically in OWL/OWL 2, enabling stronger reasoning. ⁹

Core components and building blocks

Across most ontology engineering practices, these components recur (terminology differs by language, but the conceptual roles are stable):

Classes (concepts/types) represent sets of individuals (e.g., `Person`, `Company`). ¹⁴

Properties (relations/attributes) relate individuals to individuals (object properties) or individuals to data values (data properties). ¹⁵

Individuals/instances are concrete entities in the domain being described (e.g., “Zurich”, “Invoice #123”)—sometimes stored in the same artefact as the ontology, but often separated as instance data (ABox) in a KG. ¹⁶

Axioms are logical statements that constrain meaning (e.g., subclass, equivalence, disjointness, domain/range) and enable inference. ¹⁷

Annotations/metadata (labels, definitions, provenance) are critical in practice for usability, governance, and downstream tooling—even when they do not change formal entailments. ¹⁸

Semantics and reasoning

OWL 2 provides a formally defined semantics and supplies constructs for **classes, properties, individuals, and data values**, with multiple standard documents defining its syntax and interpretation.

¹⁹ OWL’s formal semantics is closely related to description logics; the OWL 2 Direct Semantics specification explicitly positions it in this lineage. ⁸

A central modelling consequence of logic-based semantics is that many KR formalisms adopt the **open-world assumption**: knowledge bases are not assumed complete; absence of a fact does not imply its negation. This is explicitly discussed in description logic literature and is a key conceptual difference from many database-style validation mindsets. ²⁰ For “data quality” style checks (e.g., “every Customer must have exactly one birth date”), constraint languages such as SHACL define validation over RDF graphs, complementing OWL reasoning rather than replacing it. ²¹

Foundational history and standards

Historical context in brief

Ontology engineering matured at the intersection of knowledge representation, AI, and information systems. A key early influence in modern CS discourse is Thomas R. Gruber ²² 's 1993 work on portable ontology specifications, which framed ontologies as shared vocabularies enabling reuse across systems. ⁵ Later synthesis work (e.g., “formal, explicit specification of a shared conceptualisation”) consolidated earlier definitions and became widely adopted in knowledge engineering and Semantic Web communities. ⁶

The Semantic Web vision popularised ontologies as a web-scale interoperability mechanism. The well-known Scientific American article by Tim Berners-Lee ²³ and co-authors argued for web content with machine-processable meaning, accelerating standardisation and adoption efforts around RDF/OWL and linked data approaches. ²⁴

Core W3C knowledge representation and querying standards

The following standards are the backbone of many ontology + KG stacks:

RDF (Resource Description Framework) provides a graph-based data model: RDF graphs are sets of **subject-predicate-object triples**, and RDF datasets can organise multiple graphs (default + named graphs). ²⁵ Historically, RDF became a W3C Recommendation in 1999 and later evolved into RDF 1.1 (Recommendation in 2014). ²⁶

RDFS (RDF Schema) adds a lightweight schema vocabulary over RDF, providing modelling primitives such as class/property hierarchies. The RDFS 1.1 recommendation explicitly describes it as a **data-modelling vocabulary for RDF data** and as an extension of the basic RDF vocabulary. ²⁷

OWL / OWL 2 (Web Ontology Language) is a more expressive ontology language on top of RDF with formally defined semantics and explicit support for classes, properties, individuals, and data values. OWL became a W3C Recommendation in 2004; OWL 2 became a W3C Recommendation in 2009 (with a second edition in 2012). ²⁸ OWL 2 also standardises **profiles** (EL/QL/RL) intended to trade off expressivity vs reasoning performance. ²⁹

SKOS (Simple Knowledge Organization System) is designed to represent taxonomies/thesauri/classification schemes as linked data. The SKOS Reference describes SKOS as a **common data model for sharing and linking knowledge organisation systems** via the Web. ³⁰ SKOS became a W3C Recommendation in 2009. ³¹

SPARQL 1.1 is the W3C-standard query language for RDF graphs; the SPARQL 1.1 Query specification defines the syntax and semantics and highlights querying across diverse data sources, including cases where data is “viewed as RDF via middleware”. ³²

SHACL (Shapes Constraint Language) is a W3C Recommendation for validating RDF graphs against constraints expressed as RDF (“shapes graphs”), producing validation results/reports. ²¹ (Notably, SHACL continues to evolve; SHACL 1.2 Core appears as a 2026 W3C technical report.) ³³

Conceptual and practical links between ontologies, knowledge graphs, and semantic layers

Overlaps and differences

Ontologies and knowledge graphs

A practical “stack view” that matches many real deployments is:

- An **ontology** provides shared meaning: vocabularies + constraints/axioms enabling inference. ³⁴
- A **knowledge graph** provides instance-level facts (and often provenance/context), typically as a graph of entities and relations. A comprehensive tutorial paper on KGs describes their data models, querying, validation, and relationship to schema/ontologies. ³⁵

In this sense, ontology and KG are **complements**: the ontology is commonly the **schema and semantics**, while the KG is the **data** (plus potentially a schema). Many KGs are partially schema-less or use a lightweight schema; others are “ontology-rich” and rely on OWL reasoning. ³⁶

Semantic layers (two important meanings)

In enterprise practice, “semantic layer” is used in at least two ways:

1) **BI semantic layer / semantic model**: a business-facing abstraction that defines dimensions, relationships, and metrics so that downstream tools can issue consistent queries and compute consistent KPIs. Looker’s documentation states that LookML is used to create **semantic data models** describing dimensions, aggregates/calculations, and relationships, which Looker uses to generate SQL. ³⁷ The dbt Semantic Layer describes centralised metric definitions for consistent downstream consumption. ³⁸ Microsoft describes Power BI “semantic models” as data models “ready for reporting and visualization”, reflecting an official interpretation of semantic layer as the modelling surface between raw data and reporting. ³⁹

2) **Semantic layer for integration/access** (ontology-driven): ontology-based data access/integration (OBDA/OBDI) treats an ontology + mappings as a conceptual layer that enables users to query heterogeneous sources using ontology terms. The OBDA survey and the OBDI definition both frame this as a semantic paradigm for user-friendly access and integration. ⁴⁰

These two meanings share a goal (usable, consistent semantics) but differ in **formality** and typical **reasoning/querying** regime: BI semantic layers focus on metrics and business logic; OBDA semantic layers focus on conceptual interoperability and often DL-based reasoning and query rewriting. ⁴¹

Roles in data integration, querying, reasoning, and analytics

Data integration - Ontologies support integration by enforcing consistent conceptualisations across sources, often supplemented by **mappings** (e.g., OBDA/OBDI) that relate source schemas to ontology predicates. ⁴²

- Knowledge graphs often act as an integration target: extracted/linked facts from multiple systems are consolidated as a graph with stable identifiers and shared semantics. ⁴³

Querying - RDF-centric KGs use **SPARQL** for querying graphs and federating across endpoints; SPARQL 1.1 explicitly addresses querying across diverse data sources. ³²

- Property-graph KGs (common in operational graph databases) use graph query languages such as

Cypher; Neo4j's docs describe nodes, relationships, properties, and labels in the property graph model.

44

- BI semantic layers compile user questions into SQL against warehouses; Looker's docs state that Looker uses LookML models to construct SQL queries. 37

Reasoning and validation - OWL reasoning (profile-dependent) can derive implicit knowledge (classification, entailments) from axioms and facts. 9

- SHACL provides constraint-based validation of RDF graphs against shapes, commonly used as a data quality gate for KGs. 21

- Some graph platforms include built-in reasoning/materialisation; GraphDB's documentation describes forward-chaining entailment rule application and total materialisation strategies. 45

Comparison table of key attributes

Artefact	Primary purpose	Typical structure	Typical expressivity	Querying & inference	Tooling & standards alignment	Typical use cases
Ontology	Shared conceptual model; interoperability; reasoning	Schema-level classes/ properties + axioms; sometimes instances	Ranges from lightweight (RDFS/ SKOS-like) to expressive OWL 2 axioms	OWL reasoning (profile-dependent); may be paired with SHACL for validation	W3C OWL 2 documents define constructs and semantics 19 ; edited in tools like Protégé 46	Enterprise vocabularies, data integration, policy/rule modelling, semantic interoperability
Knowledge graph	Integrated factual graph of entities/ relations; search/ analytics/ context	Instance-heavy graph of nodes/ edges; may include schema/ ontology	Varies: RDF graphs + RDFS/OWL; or property graphs with application-defined constraints	SPARQL for RDF KGs 47 ; Cypher for property graphs 48 ; validation via SHACL for RDF KGs 49	KG survey emphasises data models, querying, validation, and schema interplay 50 ; interoperability between RDF and property graphs is an active topic 51	Customer 360, semantic search, recommendation, compliance linking, RAG grounding
BI semantic layer	Consistent business metrics & dimensions; governed self-service analytics	Model of tables, relationships, measures, dimensions; often warehouse-backed	Mostly arithmetic/ aggregation logic, joins, dimensions; not typically formal logic	Compiles to SQL; centralises metric definitions; limited formal inference	LookML creates semantic data models and generates SQL 37 ; dbt Semantic Layer centralises metrics 52	KPI consistency, governed metrics, self-service BI, analytics APIs

Artefact	Primary purpose	Typical structure	Typical expressivity	Querying & inference	Tooling & standards alignment	Typical use cases
Ontology-based semantic layer (OBDA/OBDI)	Conceptual querying/integration over heterogeneous sources	Ontology + mappings to sources; often virtualisation	Often DL-based (e.g., OWL profiles) to support query rewriting	Queries posed in ontology terms, rewritten to source queries	OBDA is described as semantic tech for convenient, user-friendly access ⁵³ ; OBDI definition explicitly includes ontology + sources + mapping ⁵⁴	Data virtualisation, integration across systems without centralising all data

Automated construction with NLP and LLMs

The classical “text to ontology/KG” pipeline

Ontology learning and KG construction from text are long-standing research areas; the core idea is to support ontology engineers by extracting candidate terms, relations, and constraints from corpora. ⁵⁵ A widely used decomposition (common across ontology learning literature) includes:

- 1) **Corpus preparation** (collect, clean, segment, deduplicate)
- 2) **Term/concept extraction** (candidate classes/concepts)
- 3) **Taxonomy induction** (subclass relations; concept hierarchies)
- 4) **Relation extraction** (non-taxonomic relations)
- 5) **Axiom/constraint learning** (domains/ranges, disjointness, cardinalities—harder)
- 6) **Population** (instances) and **linking** (entity resolution, linking across sources)
- 7) **Evaluation and maintenance** (iterative refinement) ⁵⁶

Two practical NLP building blocks recur in real systems:

- **Named Entity Recognition (NER)**: identifies spans corresponding to entities (people, organisations, products, etc.). spaCy documents NER as a pipeline component that predicts labelled spans and notes that performance depends on trained models and may require tuning. ⁵⁷
- **Open Information Extraction (OpenIE)**: extracts open-domain relational tuples (subject, relation, object). Stanford CoreNLP’s OpenIE documentation explicitly describes producing relation triples such as `born-in(Barack Obama, Hawaii)`. ⁵⁸

These components typically feed **entity linking** (canonical IDs for mentions) and **relation normalisation** (mapping surface relations like “was born in” to canonical predicates). ⁵⁹

Where LLMs change the picture

Recent surveys describe LLMs as shifting KG construction from rigid modular pipelines toward more unified, language-driven approaches—while still requiring strong normalisation and validation layers.

⁶⁰ A useful lens is the distinction between:

- **Schema-based LLM pipelines:** the ontology/schema is defined (or iteratively refined) and the LLM extracts facts into that schema; emphasis is on consistency and normalisation. ⁶¹
- **Schema-free (or schema-light) LLM pipelines:** the LLM proposes entities/relations first, and the schema emerges from clustering/induction; emphasis is on discovery and flexibility, but with higher governance risk. ⁶²

There is also an emerging benchmarking culture around “LLMs for ontology learning” (e.g., shared tasks evaluating extraction of taxonomy/non-taxonomic relations and axiom types), reflecting a recognition that ontology learning is not just about triples but also about **axioms** and **modelling commitments**.

⁶³

Mapping, alignment, and human-in-the-loop workflows

Mapping and alignment are often the hardest part of real deployments because heterogeneous sources and vocabularies must be reconciled. A dedicated body of work treats this as **ontology matching**, aiming to find correspondences between ontology entities to resolve semantic heterogeneity. ⁶⁴ The Ontology Alignment Evaluation Initiative ⁶⁵ has run evaluation campaigns since 2004 and provides shared test cases and evaluation modalities. ⁶⁶

In practice, most production-grade pipelines are **human-in-the-loop**: - LLM/NLP components generate candidates (classes, relations, mappings, instance facts). ⁶⁷

- Domain experts validate high-impact elements (core taxonomy, key relations, identity rules). ⁶⁸

- Automated validators (SHACL) + reasoners (OWL profiles) catch inconsistencies early. ⁶⁹

A practical enabler for safer LLM integration is **schema-constrained generation**. OpenAI ⁷⁰’s API documentation describes “Structured Outputs” as enforcing that model outputs adhere to a supplied JSON Schema, reducing schema-breaking outputs in extraction/population steps. ⁷¹

Evaluation metrics across the pipeline

Because ontology + KG projects have multiple artefacts (schema, data, mappings, queries), evaluation must be multi-dimensional:

Pipeline stage	What you evaluate	Common metrics/ approaches	Practical “good enough” signals
Entity extraction	Are entity mentions detected and typed correctly?	Precision/recall/F1 on annotated corpora; error analysis by entity type ⁷²	Stable performance on in-domain documents; low false positives for high-cost types
Relation/ OpenIE extraction	Are relations captured as correct tuples?	Tuple-level precision/ recall; manual audit of frequent patterns; known OpenIE trade-offs ⁷³	High recall first, then normalise; clear strategy for relation canonicalisation

Pipeline stage	What you evaluate	Common metrics/ approaches	Practical “good enough” signals
Ontology/ schema quality	Is the conceptualisation coherent and usable?	Competency questions; modelling review; pitfall scanning; consistency via reasoners ⁷⁴	Competency questions answered with simple queries; no unsatisfiable core classes
KG data quality	Are triples/edges correct and well-identified?	SHACL validation reports; sampling audits; provenance completeness ²¹	Constraint violations are explainable/actionable; identity rules prevent duplicates
Alignment/ mapping quality	Are correspondences correct and non-contradictory?	Precision/recall/F1 for alignments; benchmark using OAEI-style tracks when applicable ⁷⁵	High-confidence mappings for key entities; repair mechanisms for inconsistent maps
Query usefulness	Does the system answer priority questions?	Task success on query set; latency/cost; user acceptance tests ⁷⁶	Users can express business questions in model terms; consistent KPI answers

Tools, libraries, and platforms

The ecosystem can be grouped into (i) NLP/LLM extraction, (ii) ontology/KG modelling and validation, and (iii) storage/query platforms. The tools below were explicitly requested; where possible, the cited references are the tool’s official documentation or primary publications.

Tool comparison table

Tool	Category	What it’s commonly used for	Pros	Cons / watch-outs	Primary source
spaCy	NLP library	Tokenisation, parsing, NER pipelines feeding KG extraction	Fast, production-oriented pipelines; configurable components and training ⁷⁷	Domain adaptation often required; model quality depends heavily on training data ⁷⁸	⁷⁹
Hugging Face ⁸⁰ (Transformers)	NLP/ML framework	Model hub + libraries for QA/IE tasks used in extraction and linking	Strong ecosystem, many pretrained models and task guides ⁸¹	Model choice/ governance is non-trivial; extraction often needs post-processing	⁸²

Tool	Category	What it's commonly used for	Pros	Cons / watch-outs	Primary source
Stanford CoreNLP / OpenIE	NLP toolkit	OpenIE triples and relation extraction as candidate KG edges	OpenIE emits subject-relation-object tuples directly ⁵⁸	OpenIE relations are surface-texty; requires canonicalisation to schema predicates	⁸³
RDFLib	RDF programming library	Create/parse/query RDF graphs; SPARQL querying in Python	Pure Python; supports many RDF serialisations; SPARQL 1.1 implementation ⁸⁴	In-memory graphs can hit scale limits; production triplestores still needed for large KGs	⁸⁵
OWL API	OWL engineering library	Java API for creating/manipulating OWL ontologies; reasoner interfacing	Widely used; aligned with OWL 2; supports reasoning workflows ⁸⁶	Java ecosystem; learning curve for large axiom sets and imports	⁸⁷
Protégé	Ontology editor	Interactive OWL ontology editing, refactoring, debugging, visualisation	De facto standard editor; supports OWL 2; plugins and explanation support ⁴⁶	Manual editing doesn't scale without governance; collaborative workflows need process/tooling	⁴⁶
Neo4j ⁸⁸	Graph DB (property graph)	Operational knowledge graphs, graph analytics, application graph storage	Mature property graph model (nodes/relationships/properties); Cypher query language ⁴⁴	Property graphs lack a single W3C-standard semantic layer; interoperability with RDF is non-trivial but studied ⁵¹	⁴⁴
GraphDB (Ontotext)	RDF triplestore	RDF/SPARQL storage; reasoning/materialisation; KG management features	Supports RDF+SPARQL; includes rule-based reasoning/materialisation ⁸⁹	Materialisation costs must be managed; ruleset/versioning and explainability matter	⁸⁹

Tool	Category	What it's commonly used for	Pros	Cons / watch-outs	Primary source
OpenAI API	LLM platform	Assisted schema design, extraction, normalisation, validation support	Structured Outputs can enforce JSON-schema adherence for extraction/population ⁷¹	Still requires grounding, verification, and audit trails; not a replacement for constraints/reasoners	⁷¹

Notes on interoperability and technology choices

Choosing between RDF triplestores and property-graph databases is often about semantics and tooling as much as performance. A dedicated paper surveys interoperability issues between RDF and property graphs and distinguishes syntactic, semantic, and query interoperability challenges. ⁵¹ This matters because many enterprises end up with both paradigms: RDF for standards-heavy interchange and reasoning; property graphs for application-centric graph workloads. ⁹⁰

Workflows, validation, and a pilot roadmap

Workflow for building an ontology from scratch

This workflow combines practical engineering guidance (e.g., *Ontology Development 101*) with standards-aware validation:

- 1) **Scope and purpose definition:** define the domain boundary, stakeholders, and intended tasks. Capture “competency questions” (queries the ontology must support). ⁹¹
- 2) **Reuse review:** survey existing vocabularies/ontologies (including SKOS concept schemes if the domain is mostly taxonomy). Reuse reduces fragmentation. ³⁰
- 3) **Conceptual modelling:** draft core classes, relations, and key constraints; decide whether you need OWL expressivity or RDFS/SKOS is sufficient. ⁹²
- 4) **Formalisation in OWL (if appropriate):** encode classes/properties/axioms using an OWL editor such as Protégé; decide on OWL 2 profile (EL/QL/RL) if reasoning scalability is a concern. ⁹³
- 5) **IRI/namespace strategy and annotations:** establish stable identifiers and human-friendly labels/definitions. (SKOS-style labels/notes can be useful even for OWL ontologies.) ⁹⁴
- 6) **Reasoning & consistency checks:** run a reasoner (profile-appropriate) to detect unsatisfiable classes and unintended inferences. ⁹
- 7) **Constraint layer (optional but common):** define SHACL shapes for integrity constraints that are operationally required at ingestion time (e.g., required properties, cardinalities) and run SHACL validation in CI. ²¹
- 8) **Release & governance:** version the ontology; document modelling decisions; establish change control and deprecation patterns. Iteration is expected. ⁹⁵

Example “competency question” prompt (for human workshop or LLM-assisted ideation)

You are helping design an OWL ontology for a customer support domain.

Task:

- 1) Propose 12 competency questions that the ontology must support.
- 2) For each question, list the key entities (classes) and relations (properties) that must exist.
- 3) Mark questions as: MUST / SHOULD / COULD.

Constraints:

- Use business language (not OWL syntax).
- Avoid implementation-specific terms (tables, columns).
- Keep each question answerable from a knowledge graph.

Workflow for extracting a knowledge graph from corpora

A robust corpus-to-KG workflow emphasises repeatability and validation gates:

- 1) **Corpus selection and governance:** define document types, licences, and update cadence; create a “gold” evaluation subset. ⁹⁶
- 2) **Text processing pipeline:** sentence splitting, tokenisation, normalisation; choose an NLP stack. spaCy provides configurable processing pipelines; Stanford CoreNLP provides OpenIE. ⁹⁷
- 3) **Entity extraction:** NER + domain-specific dictionaries; optionally fine-tune models for in-domain entity types. ⁵⁷
- 4) **Relation extraction:** OpenIE to get candidate tuples quickly, or supervised relation extraction when relation inventory is known. ⁸³
- 5) **Entity resolution & linking:** canonicalise strings into stable IDs; merge duplicates; attach provenance (document ID, span offsets, confidence). ⁹⁸
- 6) **Schema alignment:** map extracted relations to ontology predicates (or induce a schema if starting schema-free). Ontology matching techniques and evaluation initiatives (OAEI) are relevant when aligning to existing ontologies. ⁹⁹
- 7) **Serialisation and load:** represent the KG in RDF (triples/quads) if SPARQL + W3C semantics are priorities; consider property graphs for application-centric usage. RDFLib can build/query RDF in Python; triplestores/graph DBs handle scale. ¹⁰⁰
- 8) **Validation:** SHACL constraints for ingestion checks; OWL reasoning (when you have an ontology) for semantic consistency; measure extraction precision/recall on the gold set. ¹⁰¹
- 9) **Query productisation:** publish example SPARQL/Cypher queries and dashboards; enforce metric/definition consistency if the KG feeds BI. ¹⁰²

Concrete mini-example (from text to triples)

Suppose you extract: “Acme Ltd acquired Beta GmbH in 2022.”

A normalised RDF-style representation (simplified) might be:

```
@prefix ex: <http://example.org/> .  
  
ex:acme ex:acquired ex:beta .  
ex:acme ex:acquisitionYear "2022" .
```

RDF's triple-based graph structure is the underlying model here. ¹⁰³

Workflow for using LLMs to assist schema design and population

A safe pattern is to use LLMs for **candidate generation** and **normalisation support**, while enforcing correctness through constraints and review.

- 1) **Provide the modelling contract:** give the LLM the ontology (or a schema skeleton) and an explicit output schema (JSON Schema for extraction results). ¹⁰⁴
- 2) **Extract with constrained output:** require the LLM to emit only what your pipeline can validate (entities with IDs, relations drawn from an allowed set, evidence spans). ⁷¹
- 3) **Post-process deterministically:** canonicalise labels; enforce URI policies; reject unknown predicates; require provenance fields. ¹⁰⁵
- 4) **Validate:** run SHACL (data constraints) and OWL reasoning (semantic constraints) where applicable; surface violations as feedback to the extraction step. ¹⁰⁶
- 5) **Human review loops:** route low-confidence or high-impact changes (new classes, identity rules, key mappings) to ontology engineers/domain experts. ¹⁰⁷
- 6) **Evaluate continuously:** maintain a benchmark set of documents and expected triples/axioms; track extraction metrics and validation error rates release-to-release. ¹⁰⁸

Example extraction prompt (schema-based, with evidence)

System role: You are an information extraction module for building a knowledge graph.

Input:

- Ontology snippet (classes and allowed predicates):
Allowed entity types: Person, Organisation, Product, Contract
Allowed predicates:
 employedBy(Person -> Organisation)
 acquired(Organisation -> Organisation)
 signedContract(Person -> Contract)
 contractValue(Contract -> xsd:decimal)

Task:

From the document below, extract a JSON object with:

- entities: [{temp_id, type, label, evidence_span}]
- relations: [{subj_temp_id, predicate, obj_temp_id, evidence_span}]

Rules:

- Use only the allowed types and predicates.
- If uncertain, omit rather than guess.
- Evidence span must be an exact substring from the document.

Document:

<<<paste document text>>>

Prompt engineering tips (practical, validation-oriented)

Use a “contract-first” style where the LLM is boxed into (i) a fixed predicate set, (ii) evidence requirements, and (iii) a strict output schema—then treat the LLM as a probabilistic proposer and your validators as the final gate. This approach is aligned with structured-output guidance that enforces JSON-schema adherence. ¹⁰⁹

Recommended evaluation checklist and best practices

A compact checklist that mirrors what tends to fail in pilots:

Scope, users, and success criteria - A written set of competency questions and acceptance tests for priority queries. ⁹¹

- A decision on whether the primary consumer is search/RAG, integration, BI metrics, or reasoning (these drive language/tool choices). ¹¹⁰

Modelling quality - Clear modelling principles (naming, class/property patterns, identity strategy). ⁹⁵

- OWL profile choice justified by reasoning needs (EL/QL/RL where appropriate). ¹¹¹

Data quality and validation - SHACL shapes for ingestion constraints; CI gate on SHACL validation reports. ²¹

- Reasoner checks for logical consistency and unintended subsumptions. ¹¹²

- Provenance captured for edges/nodes (document/source, timestamp, confidence). ⁵⁰

Extraction and alignment quality - Gold evaluation set; precision/recall tracked for core entities/relations. ¹¹³

- Alignment evaluated (where relevant) against established methodologies/benchmarks (OAEI-style thinking). ⁷⁵

Governance and lifecycle - Versioning of ontology and shapes; deprecation policy; release notes. ¹¹⁴

- Human review workflows for schema changes and identity rules. ¹¹⁵

Pilot project roadmap

Below is a small pilot plan (roughly 6 weeks) suitable for a first ontology + KG slice (one domain, one or two corpora/sources, a limited query set). The sequencing reflects common methodology guidance that ontology development is iterative and that validation should be built in early. ⁹⁵

```
gantt
    title Ontology + Knowledge Graph pilot (6 weeks)
    dateFormat YYYY-MM-DD
    axisFormat %d %b

    section Discovery
        Scope, users, competency questions           :a1, 2026-03-03, 7d
        Source inventory + corpus sampling           :a2, 2026-03-05, 7d

    section Ontology & constraints
        Draft conceptual model (classes/relations)    :b1, 2026-03-10, 7d
        OWL encoding + annotations (Protégé/OWL API):b2, 2026-03-13, 10d
        SHACL shapes for ingestion constraints        :b3, 2026-03-17, 7d

    section Extraction & KG build
        NLP/LLM extraction prototype (NER/OpenIE)    :c1, 2026-03-17, 10d
        Normalisation + entity linking + provenance  :c2, 2026-03-24, 7d
        Load into store + query endpoints            :c3, 2026-03-26, 7d
```

section Validation & iteration

Reasoning + SHACL validation gates :d1, 2026-03-27, 7d

Evaluation on gold set + error analysis :d2, 2026-03-31, 7d

Demo: priority queries + BI/semantic layer fit :d3, 2026-04-04, 7d

The core standards and methods referenced in this roadmap—RDF graph modelling, OWL semantics and profiles, SPARQL querying, and SHACL validation—are explicitly defined in W3C recommendations and supporting documentation. ¹¹⁶

¹ ⁵ ³⁴ A Translation Approach to Portable Ontology Specifications

https://tomgruber.org/writing/ontologia-kaj-1993.pdf?utm_source=chatgpt.com

² ²⁵ ¹⁰³ ¹¹⁶ RDF 1.1 Concepts and Abstract Syntax

https://www.w3.org/TR/rdf11-concepts/?utm_source=chatgpt.com

³ ³⁵ ³⁶ ⁴³ ⁵⁰ ⁷⁰ ⁹⁸ ¹¹⁰ Knowledge Graphs

https://aidanhogan.com/docs/knowledge-graphs-computing-surveys.pdf?utm_source=chatgpt.com

⁴ ⁶⁰ ⁶⁷ ¹⁰⁷ LLM-empowered knowledge graph construction: A survey

https://arxiv.org/pdf/2510.20345?utm_source=chatgpt.com

⁶ Paper Knowledge engineering: Principles and methods

https://www.sciencedirect.com/science/article/abs/pii/S0169023X97000566?utm_source=chatgpt.com

⁷ What is an ontology

https://iaoa.org/isc2012/docs/Guarino2009_What_is_an_Ontology.pdf?utm_source=chatgpt.com

⁸ ⁹ ²² ¹¹² OWL 2 Web Ontology Language Direct Semantics (Second ...

https://www.w3.org/TR/owl2-direct-semantics/?utm_source=chatgpt.com

¹⁰ ¹¹ ¹² Ontologies: Principles, Methods and Applications - AIAI

https://www.aiai.ed.ac.uk/project/oplan/documents/1996/96-ker-intro-ontologies.pdf?utm_source=chatgpt.com

¹³ ²⁷ RDF Schema 1.1

https://www.w3.org/TR/rdf-schema/?utm_source=chatgpt.com

¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁹ ²³ ⁹² OWL 2 Web Ontology Language Structural Specification ...

https://www.w3.org/TR/owl2-syntax/?utm_source=chatgpt.com

¹⁸ ³⁰ ⁹⁴ SKOS Simple Knowledge Organization System Reference

https://www.w3.org/TR/skos-reference/?utm_source=chatgpt.com

²⁰ An Introduction to Description Logics

https://assets.cambridge.org/97805217/81763/excerpt/9780521781763_excerpt.pdf?utm_source=chatgpt.com

²¹ ⁴⁹ ⁶⁹ ¹⁰¹ ¹⁰⁶ Shapes Constraint Language (SHACL)

https://www.w3.org/TR/shacl/?utm_source=chatgpt.com

²⁴ Scientific American: Feature Article: The Semantic Web

<https://www-sop.inria.fr/acacia/cours/essi2006/>

[Scientific%20American_%20Feature%20Article_%20The%20Semantic%20Web_%20May%202001.pdf?](#)

[utm_source=chatgpt.com](#)

²⁶ Resource Description Framework (RDF) Model and Syntax ...

https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/?utm_source=chatgpt.com

²⁸ OWL Web Ontology Language Overview

https://www.w3.org/TR/owl-features/?utm_source=chatgpt.com

- 29 111 OWL 2 Web Ontology Language Profiles
https://www.w3.org/2009/pdf/REC-owl2-profiles-20091027.pdf?utm_source=chatgpt.com
- 31 SKOS Simple Knowledge Organization System Reference ...
https://www.w3.org/standards/history/skos-reference/?utm_source=chatgpt.com
- 32 47 76 102 SPARQL 1.1 Query Language
https://www.w3.org/TR/sparql11-query/?utm_source=chatgpt.com
- 33 SHACL 1.2 Core
https://www.w3.org/TR/shacl12-core/?utm_source=chatgpt.com
- 37 Introduction to LookML | Looker
https://docs.cloud.google.com/looker/docs/what-is-lookml?utm_source=chatgpt.com
- 38 52 dbt Semantic Layer | dbt Developer Hub - dbt Labs
https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl?utm_source=chatgpt.com
- 39 Semantic Models in the Power BI Service
https://learn.microsoft.com/en-us/power-bi/connect-data/service-datasets-understand?utm_source=chatgpt.com
- 40 41 53 96 Ontology-Based Data Access: A Survey
https://www.dcs.bbk.ac.uk/~roman/papers/OBDA-IJCAI-18.pdf?utm_source=chatgpt.com
- 42 54 105 Ontology-Based Data Access and Integration
https://www.inf.unibz.it/~calvanese/papers/calv-et-al-EDBS-2017.pdf?utm_source=chatgpt.com
- 44 Graph database concepts - Getting Started
https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/?utm_source=chatgpt.com
- 45 Reasoning — GraphDB 11.3 documentation
https://graphdb.ontotext.com/documentation/11.3/reasoning.html?utm_source=chatgpt.com
- 46 93 protégé - Stanford University
https://protege.stanford.edu/?utm_source=chatgpt.com
- 48 What is Cypher - Getting Started
https://neo4j.com/docs/getting-started/cypher/?utm_source=chatgpt.com
- 51 90 RDF and Property Graphs Interoperability: Status and Issues
https://ceur-ws.org/Vol-2369/paper01.pdf?utm_source=chatgpt.com
- 55 56 Ontology Learning and Population from Text
https://content.e-bookshelf.de/media/reading/L-1458-03466a3f87.pdf?utm_source=chatgpt.com
- 57 EntityRecognizer · spaCy API Documentation
https://spacy.io/api/entityrecognizer?utm_source=chatgpt.com
- 58 59 73 83 OpenIE - CoreNLP
https://stanfordnlp.github.io/CoreNLP/openie.html?utm_source=chatgpt.com
- 61 62 LLM-empowered knowledge graph construction: A survey
https://arxiv.org/abs/2510.20345?utm_source=chatgpt.com
- 63 108 LLMs4OL 2025 Overview: The 2nd Large Language ...
https://www.tib-op.org/ojs/index.php/ocp/article/view/2913?utm_source=chatgpt.com
- 64 99 Jérôme Euzenat Pavel Shvaiko - Ontology Matching - eBooks
https://content.e-bookshelf.de/media/reading/L-2678376-ed6ba6c8a8.pdf?utm_source=chatgpt.com
- 65 72 78 113 spaCy 101: Everything you need to know
https://spacy.io/usage/spacy-101?utm_source=chatgpt.com

66 **Ontology Alignment Evaluation Initiative::2025**

https://oaei.ontologymatching.org/2025/?utm_source=chatgpt.com

68 74 80 88 91 95 114 **Ontology Development 101: A Guide to Creating Your First ...**

https://protege.stanford.edu/publications/ontology_development/ontology101.pdf?utm_source=chatgpt.com

71 104 109 **Structured model outputs | OpenAI API**

https://developers.openai.com/api/docs/guides/structured-outputs/?utm_source=chatgpt.com

75 **Ontology Alignment Evaluation Initiative::Home**

https://oaei.ontologymatching.org/?utm_source=chatgpt.com

77 79 97 **Language Processing Pipelines**

https://spacy.io/usage/processing-pipelines?utm_source=chatgpt.com

81 82 **Question answering**

https://huggingface.co/docs/transformers/en/tasks/question_answering?utm_source=chatgpt.com

84 85 100 **RDFLib - Read the Docs**

https://rdflib.readthedocs.io/?utm_source=chatgpt.com

86 87 **owlcs/owlapi: OWL API main repository**

https://github.com/owlcs/owlapi?utm_source=chatgpt.com

89 **What is GraphDB? — GraphDB 11.3 documentation**

https://graphdb.ontotext.com/documentation/11.3/index.html?utm_source=chatgpt.com

115 **Methontology**

https://oa.upm.es/5484/1/METHONTOLOGY_.pdf?utm_source=chatgpt.com